BRAC University

Department of Computer Science and Engineering

Group Assignment 1

Course Code: CSE341

Course Name: Microprocessors

Group: 05

Semester: Summer 2026

Group Members

Amirun Nahin	23201416
Sultan Mohammad Farid	23201107
Fatima Anower	20301260

Submitted To

Farzana Reefat Raha

Lecturer

Department of Computer Science and Engineering

Brac University

Submission Date: July 21, 2026

Question 1 [7 marks]

23201416 - Amirun Nahin

Topic: Addressing Modes, Machine Codes

During a lab session, your classmate Rauf is confused about 8086 machine code. He insists that because the architecture allows for a maximum of 6 bytes, every single machine code instruction must be exactly 6 bytes long. He is specifically confused as to why the machine code **8B905001h** is only 4 bytes. [Opcode for MOV instruction is 100010]

MOD / RM	00	01	10	11	
				W = 0	W = 1
000	[BX] + [SI]	[BX] + [SI] + d8	[BX] + [SI] + d16	AL	AX
001	[BX] + [DI]	[BX] + [DI] + d8	[BX] + [DI] + d16	CL	CX
010	[BP] + [SI]	[BP] + [SI] + d8	[BP] + [SI] + d16	DL	DX
011	[BP] + [DI]	[BP] + [DI] + d8	[BP] + [DI] + d16	BL	BX
100	[SI]	[SI] + d8	[SI] + d16	AH	SP
101	[DI]	[DI] + d8	[DI] + d16	CH	BP
110	d16 (direct address)	[BP] + d8	[BP] + d16	DH	SI
111	[BX]	[BX] + d8	[BX] + d16	BH	DI

1. (a) Decode the machine code into its corresponding assembly instruction. **[2]**

   (b) Briefly explain why this specific instruction requires exactly 4 bytes of machine code instead of the maximum 6 bytes. **[1]**

2. Identify the specific data addressing mode being used by the memory operand in your decoded instruction. Provide proper reasoning for your answer. **[2]**

3. Write a modified version of this assembly instruction (accessing the exact same memory location) that would require the maximum 6-byte machine code length, and identify the addressing mode of its new source operand. **[2]**

Question 2 [7 marks]

23201107 - Sultan Mohammad Farid

Topic: Introduction to Microprocessors and Microcomputers, Internal Architecture and Flags

Your classmate Fuad is tracing through this code on an 8086, where [SI] currently holds 7A2Fh:

```
MOV SI, 0300h
MOV AX, [SI]      ; loads 7A2Fh from memory
ADD AX, 1234h
```

He has two separate points of confusion:

- *"Reading [SI] from memory should be exactly as fast as reading a register like AX directly — they're both just data storage on the same chip."*
- *"Since this addition doesn't carry past FFFFh, the Overflow Flag has to stay 0."*

Answer the following questions based on the scenario -

1. (a) Explain why fetching the operand via [SI] from external memory takes more clock cycles than an equivalent register-to-register operation would, correcting Fuad's first assumption. **[1]**

   (b) Name the two system buses actively used during this memory fetch, and state what travels on each. **[1]**

2. Compute the result of ADD AX, 1234h and determine CF, ZF, SF, PF, AF, and OF. Show your reasoning, and use the result to directly correct Fuad's second assumption. **[3]**
3. Fuad rewrites the code to preload the memory value into BX first, then runs

   ADD AX, BX

   instead of adding directly from [SI]. Explain whether this changes any of the flag results from part 2, and whether it removes the timing difference identified in part 1(a).

Question 3 [7 marks]

20301260 - Fatima Anower

Topic: Memory Addressing, Addressing Modes

1. What is memory addressing? Explain its purpose with suitable examples. **[7]**

Answer 1

1. (a) Byte 1 (8Bh): In binary, this is 1000 1011. The opcode 100010 indicates a MOV instruction. The direction bit D=1 means the register specified in the reg field is the destination. The size bit W=1 means it's a 16-bit (word) operation.

Byte 2 (90h): In binary, this is 1001 0000. mod = 10 indicates memory addressing with a 16-bit displacement. REG = 010 specifies the DX register (since W=1). R/M = 000 specifies the base and index combination of [BX + SI].

Bytes 3 & 4 (50h 01h): This is the 16-bit displacement. Due to the little-endian architecture of the 8086, the bytes are stored in reverse order. This evaluates to 0150h.

Decoded Instruction:

MOV DX, [BX + SI + 0150h]

(b) This specific instruction only requires 4 bytes because it consists of 1 byte for the Opcode/D/W, 1 byte for the MOD/REG/R/M, and 2 bytes for the 16-bit displacement. The 6-byte maximum is for the longest possible 8086 instruction format in general, not for this specific MOV form. So, It does not reach the 6-byte maximum because this specific instruction does not require an additional 16-bit immediate data value to be moved into that memory location.

2. Here, Base-relative-plus-index addressing is used.

The memory operand [BX + SI + 0150H] contains a base register (BX), an index register (SI), and a displacement (0150H) combined in the offset part of the source.

3. Modified Instruction: MOV [BX + SI + 0150H], 1234H

New Source Addressing Mode: Immediate addressing.

The source operand is now a direct 16-bit hexadecimal value (1234H) provided directly within the instruction itself.

Answer 2

1. (a) Registers such as **AX** are located inside the CPU, so the processor can access them directly in a single internal operation. However, **[SI]** refers to a memory location (DS:0300h), so the CPU must communicate with external memory. This requires placing the memory address on the address bus, waiting for the memory to respond, and then reading the data through the data bus. These extra bus operations require additional clock cycles, making memory access slower than register access.

   (b) Address Bus: The memory address (DS:0300h) sent from the CPU to memory.

   Data Bus: The data value (7A2Fh) transferred from memory back to the CPU.

2. 7A2Fh + 1234h = 8C63h

   So, AX = 8C63h

   Carry Flag (CF): No carry out of bit 15. **CF = 0**

   Zero Flag (ZF): Result is 8C63h, not zero. **ZF = 0**

   Sign Flag (SF): MSB of 8C63h is 1, indicating a negative value in signed arithmetic.

   SF = 1

   Parity Flag (PF): Lowest byte = 63h

   63h = $(01100011)_2$

   Number of 1s = 4 (even). **PF = 1**

   Auxiliary Carry (AF): Lower nibble: F + 4 = 13h

   Carry occurs from bit 3 to bit 4. **AF = 1**

   Overflow Flag (OF): Signed interpretation:

   7A2Fh = +31279

   1234h = +4660

   Mathematical sum:

   31279 + 4660 = 35939

   The maximum signed 16-bit value is **32767**, so the result cannot be represented in a signed 16-bit integer.

   The computed result (8C63h) has its sign bit set, making it appear negative.

   So, **OF = 1**

Therefore, CF = 0, ZF = 0, PF = 1, AF = 1, OF = 1

And thus, Fuad's assumption is **incorrect**. The Overflow Flag is not determined by whether there is a carry out of FFFFh. Instead, it indicates signed overflow. Here, two positive numbers (7A2Fh and 1234h) produce a result (8C63h) that is negative in signed 16-bit representation, so OF = 1 even though CF = 0.

3. The flag values do not change if preloading the memory value into BX before the addition is done. The ADD instruction performs the same arithmetic on the same operands, so the result (8C63h) and all flags (CF=0, ZF=0, SF=1, PF=1, AF=1, OF=1) remain identical.

   The timing difference during the addition is removed because ADD AX, BX is a register-to-register operation, which is faster than accessing memory during the ADD.

   However, the memory must still be read once when executing:

   MOV BX, [SI]

   Therefore, the memory access cost is not eliminated. It is only moved to an earlier instruction. The ADD itself becomes faster, but the program still incurs the memory fetch overhead once to load the data into BX.

Answer 3

topic (Memory Addressing).

What is memory addressing ? Explain its purpose with a suitable example

Memory addressing is the method by which a computer identifies and accesses a specific location in memory (RAM). Every memory location has a unique number called a memory address. The CPU uses these addresses to read data from memory or write data into memory.

Purpose of Memory Addressing.

Memory addressing is important because it —

- Helps the cpu find the exact location of data or instructions.
- Allows data to be stored and retrieved efficiently.
- Enables programs to accese variables, arrays and other information correctly.
- Ensures that each memory location can be uniquely identified.

Example.

Suppose a Computer's memory contains the following data.

Memory Address	Data stored.
1000	25
1001	42
1002	17.

if the CPU needs the value 42, it accesses memory address 1001. the address tells the CPU exactly where the required data is stored.

Working process.

1. The CPU generates the required memory address.
2. The address is sent to the memory unit.
3. The memory locates the corresponding address.
4. The requested data is either read from memory or written to memory.

Conclusion: Memory addressing is a fundamental concept in computer systems. it enables the CPU to locate, store and retrieve ad data accurately by using unique memory addresses. Without memory addressing, a computer would not be able to access data efficiently. making programm execution impossible.